

이명욱

Robot Software Engineer

AMR | Embedded | FMS

STM32로 센서와 모터를 다루는 일에서 시작했습니다. 지금은 ROS 2 로봇과 다중 로봇 관제 사이의 통신, 주행, 작업 흐름을 연결하고 있습니다.

문제가 생기면 먼저 로그를 봅니다. 재현 조건을 찾고 실제 장치에서 다시 움직여 보며 원인을 좁힙니다.



사용 기술

Robot Software ROS2 Humble / Jazzy Nav2 / SLAM Toolbox / AMCL 자율주행과 위치 추정	Embedded Control STM32F407 / STM32H743 FreeRTOS / UART / I2C / DMA / PWM 센서 처리와 모터 제어
Perception & Sensors 2D LiDAR / Camera / IMU OpenCV / Camera Calibration 센서 융합과 객체 인식	Communication & Tools Zenoh / MQTT C/C++ / Python / Linux / Git 로봇 통신과 시스템 통합

학력 및 현재 과정

2026.04 – 11 예정 / 수강 중 | 대한상공회의소 서울기술교육센터 | AI 융합 로봇 SW 개발자 과정 3기
학사 | 인덕대학교 정보통신공학과 | 정보통신공학 학사 / 임베디드 제어와 로봇 프로젝트 경험
현재 개발 분야 | AMR와 로봇 시스템 통합 | ROS2 로봇과 다중 로봇 관제 사이의 통신 및 상태 흐름

프로젝트

AX Logistics Multi Robot | GoSung | AIMing Project | Tracking Patrol Robot | VIP Wearable | MPS 창고 적재 제어

GitHub / Ugie01

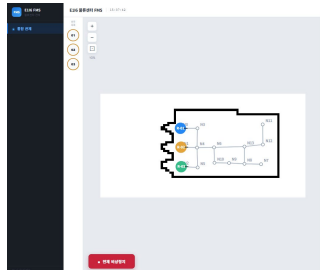
github.com/Ugie01

AX Logistics Multi Robot

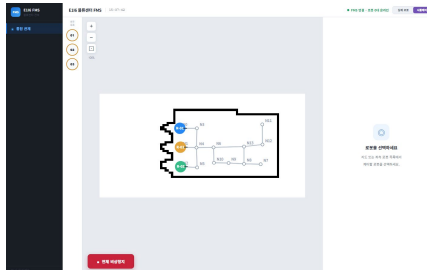
Zenoh와 MQTT로 로봇 3대와 로봇팔 작업을 연결한 ROS 2 물류 관제 시스템

2026.09 – 진행 중 | 7명 | ROS 2 Jazzy / Zenoh / MQTT / Nav2 / FastAPI

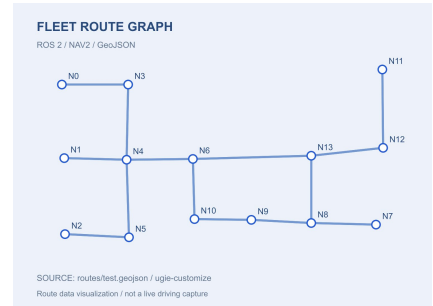
확인 결과 | 로봇 3대 주행과 Pickup → ArUco → OMX 작업 흐름 연동



대표 이미지



실제 맵과 로봇 주행 이미지 교체 예정
(현재: 모의 관제 화면)



전체 관제, 로봇, 로봇팔 구조 이미지 교체 예정 (현재: 경로 그래프)

목표와 구현

ROS 2 Jazzy 기반 로봇 3대와 OMX 로봇팔의 상태를 관제하고 주문 작업을 이어 주는 7인 팀 프로젝트입니다. 로봇과 FMS는 Zenoh, 로봇팔과 FMS는 MQTT로 통신합니다. 실제 로봇 주행과 시뮬레이션 모드를 분리하고, 주문부터 Pickup과 Workstation까지의 흐름을 통합하고 있습니다.

담당 역할 FMS 백엔드와 다중 로봇 통신, 주행, 작업 시퀀스 통합

내가 구현한 일

- 로봇 3대에 동일한 실행 환경을 구성하고 namespace 인자로 구분; 로컬 DDS와 Zenoh Bridge와 Router 연결 관리
- FMS 백엔드에서 로봇과 OMX 객체별 연결 상태와 명령을 관리하고 실기와 시뮬레이션 모드 분리
- Route Graph 경로를 Nav2 FollowWaypoints로 전달하고 주행 상태, 경로 재생성, 정지, 비상정지 제어
- Pickup, Workstation, 충전 노드별 최종 Yaw를 지정하고 일반 이동과 충전과 자동 도킹 명령 분리
- 주문 → Pickup 이동 → ArUco 정렬 → MQTT OMX 작업 명령으로 이어지는 시퀀스와 예외 흐름 통합
- Chrony로 Main PC와 로봇 시각을 맞추고 LiDAR Scan 지연을 점검해 source_timeout을 0.2초에서 0.5초로 조정

시스템 구조와 흐름

FMS 백엔드 → Zenoh / 로봇 3대 → Pickup과 ArUco → MQTT / OMX

협업 범위

경로 탐색, ArUco 정렬, 로봇팔 기능을 FMS 백엔드의 주행과 작업 시퀀스에 연결하고, 로봇과 로봇팔의 통신 환경을 구성했습니다. 제 담당 범위는 백엔드와 실기 통합입니다.

문제 해결과 결과

실행환경에서 LiDAR Scan 지연, 특수 노드 도착 방향 오류, 일반 이동 중 불필요한 자동 도킹이 발생했습니다. Main PC 기준 Chrony 동기화와 source_timeout 조정으로 지연을 완화했습니다. 특수 노드에 고정 Yaw를 넣고, 충전 명령과 저전력 자동 복귀에만 도킹이 실행되도록 조건을 분리했습니다.

로봇의 작업 지점 정지 방향과 충전 조건을 명확히 하고, Pickup 도착부터 ArUco 정렬과 MQTT 로봇팔 명령까지 하나의 작업 흐름으로 연결했습니다.

검증 범위와 한계

실환경 작업 흐름을 통합 중인 프로젝트입니다.

GoSung

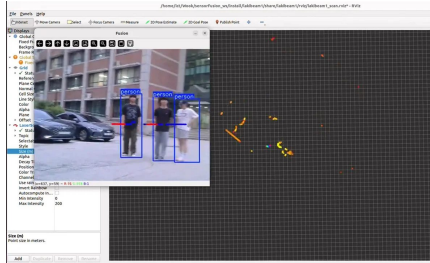
STM32 센서와 구동부와 ROS 2를 연결한 교내 배달 로봇 시제품

2024.04 – 2024.12 | 6명 | STM32F407 / FreeRTOS / ROS 2 Humble / UART / OpenCV

확인 결과 | 센서와 구동 연동 시험, 목표한 배달 주행은 미달



대표 이미지



카메라와 LiDAR 센서 융합 실험 화면



경사로 정지 시험 당시 로봇

목표와 구현

교내 카페 주문을 가정한 별도 시험 앱, DB, ROS 2 Humble과 4륜 로봇을 연결했습니다. 센서 수집과 구동부 제어는 구현했지만, 모터 토크와 마찰력과 스키드 스티어링 구조를 충분히 고려하지 못해 목표한 회전과 배달 주행은 달성하지 못했습니다.

담당 역할 STM32 센서와 구동 펌웨어와 ROS 2 연동

내가 구현한 일

- 핀 수 제약으로 STM32F407 센서 MCU와 구동 MCU를 분리하고 IMU와 GPS 데이터를 UART로 수집과 전달
- 구동 MCU에서 좌우 목표 속도(mm/s)를 받아 BLDC 모터 PWM을 출력하고 홀센서 인터럽트로 속도 변화를 추정
- 초음파 센서의 근접 정지 명령은 ROS 2 제어 경로와 별도로 구동 MCU에서 처리
- 정지 뒤 홀센서 변화로 경사로 밀림 방향을 추정하고 반대 방향 PWM을 작게 가하는 정차 보정 실험
- 카메라 보정과 YOLOv5 ROS 2와 Lakibeam 드라이버를 이용한 객체 영역 기반 LiDAR Scan 필터링 실험

시스템 구조와 흐름

IMU / GPS → 센서 STM32 → ROS 2 → 구동 STM32 / BLDC

협업 범위

Nav2, YOLOv5 ROS 2 wrapper와 Lakibeam ROS 2 Driver를 적용했습니다. 저는 MCU 펌웨어와 센서와 구동 연결, Camera와 LiDAR 데이터 결합 실험을 담당했습니다. 속도 보정은 단순 PWM 증감으로 PID 제어가 아닙니다.

문제 해결과 결과

경사로 정지 후 로봇이 미끄러졌고, 4륜 스키드 스티어링의 회전 요구 토크를 충족하지 못했습니다. 홀센서 카운트 변화와 방향을 이용해 정지 시 반대 방향 PWM을 주는 보정을 실험했습니다. 카메라와 LiDAR 데이터도 좌표 변환해 합쳤지만 시각 동기화는 적용하지 않았습니다.

센서와 구동 MCU와 ROS 2 간 데이터 흐름 및 경사로 정차 보정을 시험했습니다. 그러나 전체 배달 경로 주행은 성공하지 못했습니다.

검증 범위와 한계

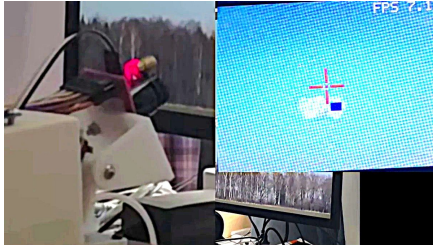
평지에서 외력으로 밀어도 경사로로 오인식했고, 속도 측정 정확도와 PWM 보정의 정량 검증과 간헐적 급가속 문제 해결은 완료하지 못했습니다. 객체 기반 Scan 필터는 객체 검출에 지나치게 의존했습니다.

AIMing Project

STM32H743 한 대에 카메라, AI 추론, Pan/Tilt 추적을 연결한 PoC

2026.07.23 – 08.04 | 4명 | STM32H743 / FreeRTOS / DCMI / DMA / Edge Impulse / C / C++

확인 결과 | 약 7 FPS 설정에서 보드 추론과 추적 흐름 확인



대표 이미지



영상 입력에서 나타난 청색 노이즈



Lock-On 상태를 표시한 보드 화면

목표와 구현

STM32H743 한 대에서 OV2640 카메라 입력과 팀원이 학습한 AI 모델의 추론 결과를 받아 Pan/Tilt 구동과 레이저 출력으로 이어 주는 시제품입니다. FreeRTOS 태스크로 영상 처리와 제어 작업을 나누었습니다.

담당 역할 OV2640과 Edge Impulse 연동과 추적 상태 로직 구현

내가 구현한 일

- 오픈소스 OV2640 설정 코드를 통합하고 카메라 프레임을 팀원의 LCD 출력 형식에 맞게 변환
- Edge Impulse FOMO 모델의 보드 추론 경로를 연결하고 얼굴 인식 모델로 연동 가능성 시험
- 추론 좌표의 흔들림에 EMA와 Miss Tolerance를 적용해 순간 검출 누락 완화
- Idle → Tracking → Lock-on 상태 머신으로 추적과 출력 흐름 구성
- 카메라 클럭을 조정해 약 7 FPS에서 14~15 FPS까지 시험했으나 청색 노이즈로 안정적인 약 7 FPS 설정을 선택

시스템 구조와 흐름

OV2640 카메라 → FreeRTOS / 추론 → EMA와 상태 머신 → Pan/Tilt와 출력

협업 범위

AI 모델 학습, LCD 기본 코드와 서보 PID 제어는 팀원이 담당했습니다. 저는 카메라, 모델, LCD, 추적 상태 로직을 STM32H743에서 연결했습니다.

문제 해결과 결과

입력 FPS를 높이면 영상에 청색 노이즈가 생겼고, 추론 좌표가 흔들리거나 일시적으로 사라졌습니다. 배선과 접점을 점검했으나 고클럭 노이즈를 없애지 못해 기존 카메라 설정으로 돌아갔습니다. 추적 좌표에는 EMA와 Miss Tolerance를 적용했습니다.

안정적인 영상 입력, 추론, Pan/Tilt 추적 흐름을 실제 보드에서 확인했습니다.

검증 범위와 한계

고클럭 입력의 영상 노이즈는 해결하지 못해 약 7 FPS로 진행했습니다. 모델 학습과 서보 PID 제어는 제 담당 범위가 아닙니다.

Tracking Patrol Robot

Pose 기반 상황 판단과 대상 재식별을 주행 명령에 연결한 추적 로봇

2026.05.11 – 05.21 | 4명 | Raspberry Pi 5 / STM32 / Python / Bluetooth / 모바일 앱

확인 결과 | Pose 상황 판단과 대상 추적과 주행 명령 시연

상황 판단



정상



위험



낙상

목표와 구현

Raspberry Pi 5에서 팀원이 만든 비전 모델의 사람과 Pose 결과를 받아 위험 상황과 낙상 의심 대상을 판단하고, STM32 주행부에 추적 명령을 보내는 4인 팀 프로젝트입니다. 모바일 앱의 수동 제어와 PID 게인 설정, Flask 영상 확인 화면도 구성했습니다.

담당 역할 Pose 상황 판단과 대상 추적과 모바일 제어 기능 구현

내가 구현한 일

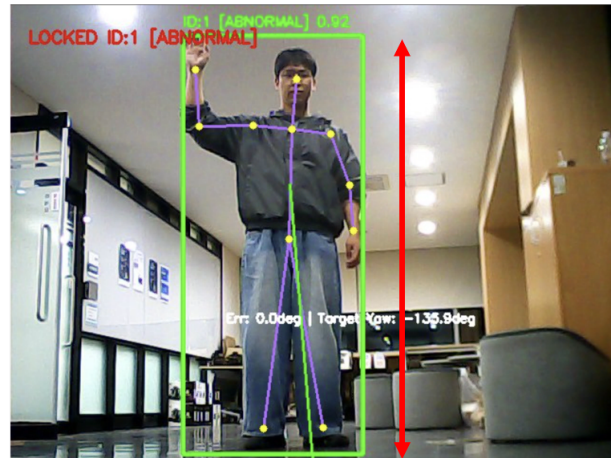
- 팔, 어깨, 다리 Pose 각도를 이용해 폭력과 낙상 의심 상황을 판단하고 추적 대상 선택
- Bounding Box의 위치와 가로와 세로 크기로 목표 방향과 화면 내 거리 유지 동작 생성
- 대상 유실 시 회전 탐색 후 색상 히스토그램을 비교해 기존 대상 재식별 시도
- 모바일 앱에서 수동 제어와 PID 게인 설정과 저장, Flask에서 카메라 영상 확인 기능 구성
- STM32 펌웨어와 PID 구현 일부를 지원하며 라즈베리파이 추적 명령과 주행부 연동

Tracking Patrol Robot

자세에 따른 거리 유지



가로 자세: 가로와 세로 점유율 중 큰 값



세로 자세: 박스 높이의 화면 점유율

낙상 상태는 바운딩 박스의 가로와 세로 점유율 중 큰 값에 별도 정지와 후진 기준을 적용합니다. 서 있는 대상은 박스 높이의 화면 점유율을 사용합니다.

시스템 구조와 흐름

Camera / Pose → 상황 판단과 재식별 → RPi 주행 명령 → STM32 / 모터

협업 범위

비전 모델 학습과 STM32 센서, 모터, PID의 주 구현은 팀원이 담당했습니다. 저는 모델 출력 후 상황 판단과 추적 로직과 앱 기능을 주로 구현했습니다.

문제 해결과 결과

상황 판단한 사람이 화면에서 사라졌다가 다시 나타나면 다른 대상으로 인식하는 문제가 있었습니다. 판단 당시 대상의 색상 히스토그램을 저장하고 재등장 객체와 비교하는 로직을 넣었습니다.

Pose 판단, 대상 추적과 주행 명령 전달을 시연했습니다.

검증 범위와 한계

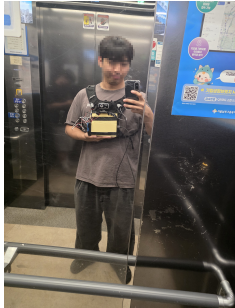
재식별은 조명과 복장 변화에 민감했습니다. 화재 감지는 시험하지 못해 범위에서 제외했고, 장시간 무인 순찰은 검증하지 않았습니다.

VIP Wearable

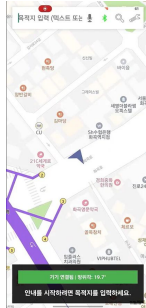
TMAP 경로, IMU 방향, AI 인식 결과를 진동으로 전달한 보행 보조 시제품

2026.07.01 – 08.12 | 2명 | Raspberry Pi 5 / UART IMU / Bluetooth Classic / TMAP API / Android Java

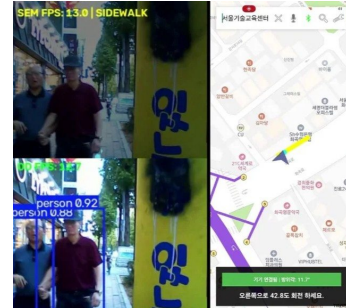
확인 결과 | 방향과 장애물 진동 연동, 전체 경로 안내는 미검증



대표 이미지



앱에서 생성한 보행 경로



장애물 인식 결과와 앱 안내 화면

목표와 구현

시각장애인의 보행 보조를 목표로 Android TMAP 경로, Raspberry Pi 5의 IMU 방향 정보와 팀원의 장애물과 보행 영역 인식 결과를 연결한 2인 시제품입니다. 목적지 입력, 방향 표시와 좌우 진동 안내를 구현했지만 GPS와 방향 오차로 전체 보행 경로의 정상 동작은 검증하지 못했습니다.

담당 역할 IMU 방향과 진동 제어와 Android 앱 기능 통합

내가 구현한 일

- 필터가 내장된 IMU의 UART Yaw를 읽어 사용자 방향을 앱에 표시하고 TMAP 경로 waypoint와 비교
- 목적지 텍스트와 음성 입력과 취소, 방향 오차 범위에 따른 직진과 회전 판단 기능 구현
- 팀원의 장애물과 보행 영역 인식 결과와 진행 방향을 서로 다른 좌우 모터 진동 패턴으로 연결
- STM32↔Raspberry Pi UART 불안정으로 Raspberry Pi 직접 제어로 설계를 바꾸고 불량 햅틱 모듈은 DC 모터로 교체
- BLE 연결 끊김이 잦아 Bluetooth Classic 연결 방식을 적용하고 후속 자동 연결 동작 구성

시스템 구조와 흐름

Android / TMAP → Bluetooth Classic → RPi / IMU와 AI → 좌우 DC 모터

협업 범위

장애물과 보행 영역 모델은 팀원이 구현했습니다. 저는 센서, 앱, 진동 동작을 연결했습니다.

문제 해결과 결과

BLE 연결과 초기 UART 구성이 불안정했고, GPS 시작 위치 오차가 커 경로 안내가 빗나갔습니다. Bluetooth Classic과 Raspberry Pi 직접 제어로 하드웨어 경로를 단순화하고 방향별 진동 패턴을 구성했습니다.

목적지 입력, 방향 표시, 장애물 및 방향별 진동 기능을 개별적으로 연동했습니다.

검증 범위와 한계

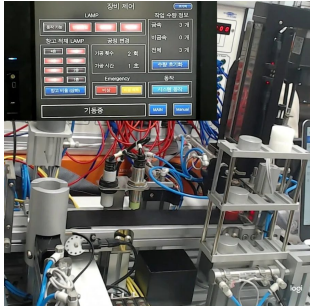
GPS, IMU, 경로 오차로 실제 목적지까지의 정상 보행 안내는 확인하지 못했습니다. 경로 음성 안내와 낙상 감지는 충분히 구현과 검증하지 못해 성과에서 제외했습니다.

MPS 창고 적재 제어

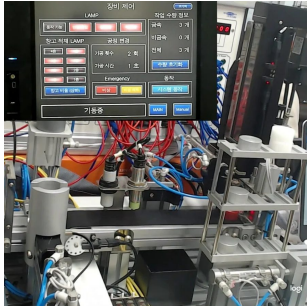
Mitsubishi PLC와 HMI로 층별 적재와 비우기, 수동 운전을 구현한 교육 프로젝트

2026.06.12 – 06.17 | 2명 | Mitsubishi PLC / GX Works2 / GT Designer3 / Ladder / HMI

확인 결과 | 층별 적재와 비우기와 HMI 수동 제어 확인



대표 이미지



MPS 트레이너 / 적재와 비우기 동작



HMI / 층별 수동 제어 화면

목표와 구현

MPS 트레이너와 Mitsubishi PLC, HMI를 이용해 창고의 층별 적재와 비우기와 수동 개별 제어를 구성한 2인 교육 프로젝트입니다.

담당 역할 PLC 시퀀스, HMI 제어 공동 구현

내가 구현한 일

- GX Works2 래더로 층별 적재와 비우기 시퀀스와 수동 개별 제어를 공동 구현
- GT Designer3 HMI에서 적재 상태 램프, 위치 버튼, 설비 점검과 티칭 화면 구성
- 3층 적재 전 4층 적재 금지, 4층 비움 전 3층 비움 금지 조건 구현
- 서보 리셋 직후 구동 오류에 대기 시간을 추가하고 OR/NOT 접점 배치 오류로 상시 도통되던 코일 수정

시스템 구조와 흐름

HMI 버튼과 램프 → PLC / Ladder → 서보와 센서 → 층별 적재와 비움

협업 범위

Mitsubishi PLC와 MPS 트레이너를 사용한 2인 공동 교육 프로젝트입니다. 시퀀스와 HMI를 함께 구현했습니다.

문제 해결과 결과

서보 모터가 리셋 직후 간헐적으로 동작하지 않았고, 래더 OR/NOT 접점의 논리 배치 오류로 출력 코일이 상시 도통됐습니다. 서보 준비 대기 시간을 넣고 자기유지와 차단 조건을 다시 확인해 래더 접점 배치를 수정했습니다.

층별 적재와 비우기, 수동 버튼과 HMI 상태 표시를 교육 장비에서 확인했습니다.

검증 범위와 한계

교육용 MPS 트레이너에서 수행한 프로젝트입니다.